

extrinsics

📌 Extrinsics estimation (외부 파라미터 추정)

1. 동기화 (Synchronization)

- **이유:** 만약 카메라 캘리브레이션에 쓸 물리적 랜드마크(체커보드, 마커 등)가 정지해 있지 않고 움직인다면, 모든 카메라를 시간적으로 정확히 맞춰야 한다.

- **방법:**

- a. 모든 비디오에서 동일한 프레임레이트(원본 fps 근처, 보통 30fps)로 프레임 추출

```
1 | ffmpeg -i VIDEO -vf "fps=30" frames/frame_%06d.jpg
```

- b. 각 영상에서 **인식하기 빠른 이벤트(예: 박수)** 를 찾아서 프레임 인덱스 차이를 보정 → 오프셋 제거 후 랜드마크 좌표들을 정렬

2. 상대 포즈 계산 (Compute relative poses)

[compute_relative_poses_robust.py](#)

- 목적: 각 카메라의 자세(Extrinsics)를 **첫 번째 카메라 기준**으로 구하기.

- **방법:**

- a. 카메라 쌍 간 상대 포즈 계산 후 **트리 구조**로 연결

- b. 최소 연결 집합(`setup.json`) 정의 필요

- ⚠ 서로 정면을 마주보는 카메라 쌍은 제외 → 기하 복원 불안정

- c. 필요한 입력 파일:

- `landmarks.json` : 각 뷰에서 검출한 랜드마크 이미지 좌표
 - `intrinsics.json` : 카메라 내부 파라미터
 - `filenames.json` : 시각화용 이미지 파일 이름

- d. 실행:

```
1 | python compute_relative_poses.py -s setup.json -i intrinsics.json -l landmarks.json -f filenames.json --dump_images
```

→ 결과: 상대 포즈(단, translation은 크기(scale) 없이 방향만 있는 단위 벡터)

- **대안 명령어 (로버스트 버전):**

- 여러 다른 뷰를 이용해 평균을 내어 더 안정적인 상대 포즈 추정

```
1 python compute_relative_poses_robust.py -s setup.json -i intrinsics.json -l landmarks.json  
-m lmeds -n 5 -f filenames.json --dump_images
```

3. 상대 포즈 연결 (Concatenate relative poses)

[concatenate_relative_poses.py](#)

- 모든 카메라 포즈를 **cam0 기준 좌표계**로 연결/체인
- 각 단계에서 **스케일을 맞추는 작업**을 수행하여 전체 일관성 확보
- 입력: **relative_poses.json** (앞 단계 결과)
- 실행:

```
1 python concatenate_relative_poses.py -s setup.json -r relative_poses.json --dump_images
```

4. 번들 조정 (Bundle adjustment)

[bundle_adjustment.py](#)

- 비선형 최소제곱(**Nonlinear Least Squares**) 으로 Intrinsics, Extrinsics, 3D 포인트 동시 최적화
- 출력 포즈는 여전히 **scale 모호성** 존재
- 입력: **poses.json** (Concatenate 단계 결과)
- 실행:

```
1 python bundle_adjustment.py -s setup.json -i intrinsics.json -e poses.json -l  
landmarks.json -f filenames.json --dump_images -c ba_config.json
```

5. 글로벌 좌표계로 변환 (Transformation to global reference system)

[global_registration.py](#)

- 번들조정 결과는 **cam0 기준 + scale 모호성** 있음 → **글로벌(월드) 좌표계**로 변환 필요
- 방법:
 - **landmarks_global.json** 파일에 최소 4개 이상의 **비대칭 랜드마크**의 실제(global) 좌표를 정의
 - 이 포인트들과 번들조정으로 얻은 3D 포인트(**ba_points.json**)의 대응점을 rigid alignment → 최적의 전역 변환 행렬 계산
 - 결과: **global_poses.json**
- 실행:

```
1 python global_registration.py -s setup.json -ps ba_poses.json -po ba_points.json -l landmarks.json -lg landmarks_global.json -f filenames.json --dump_images
```

- 만약 global landmarks가 다른 세트일 경우 → **ba_points.json** 새로 계산 필요:

```
1 python triangulate_image_points.py -p ba_poses.json -l landmarks.json --dump_images
```

전체 파이프라인 요약

1. 동기화: fps 맞춰 프레임 추출 + 이벤트(박수 등)로 오프셋 제거
2. 상대 포즈 계산: **compute_relative_poses.py** (또는 robust 버전)
3. 포즈 연결: **concatenate_relative_poses.py** → **poses.json**
4. 번들 조정: **bundle_adjustment.py** → **ba_poses.json**
5. 글로벌 변환: **global_registration.py** → **global_poses.json**

 정리하면:

- **cam0 기준 상대좌표** → **번들조정** → **월드좌표로 정렬**
- 이 과정을 거쳐야 YOLO가 뽑은 픽셀좌표를 **거실 바닥 좌표**로 변환할 수 있어.